12-Hour Acute Respiratory Failure Prediction with RD-Agent

English translation of the corresponding Chinese research report.

Abstract

This project examines the use of RD-Agent, an automated research-and-development (R&D) agent, to build a 12-hour risk-prediction model for acute respiratory failure (ARF). It independently evaluates the generated workflow through a manually developed baseline, a fair model comparison, feature ablation, multi-seed robustness experiments, feature-importance analysis, and threshold and error analysis.

This is the exploratory V1 report. The separate Final Reliability Report contains the project's current formal reliability evaluation; its results were obtained under a more stringent protocol and should not be compared directly with the V1 metrics.

The dataset contains 9,921 training samples. Each sample comprises 4,816 clinical features measured hourly over 12 consecutive hours. There are 960 positive samples (9.68%), making this a clearly class-imbalanced binary-classification task.

The project first verified that RD-Agent could run in an Apple Silicon ARM64 Ubuntu virtual machine. It then established a manually developed logistic-regression baseline and compared logistic regression with LightGBM under the same five-fold stratified cross-validation, feature set, and random-seed conditions.

A fair comparison shows that:

- Logistic-regression pooled OOF AUROC: 0.6978
- Logistic-regression pooled OOF AP: 0.2332
- LightGBM pooled OOF AUROC: 0.7770
- LightGBM pooled OOF AP: 0.3173

Under this experiment's data, feature set, and validation design, LightGBM's pooled OOF AUROC is 0.0792 higher than that of logistic regression.

Feature ablation further found that LightGBM using only 12-hour sum features performed no worse than the full feature scheme containing nine types of summary statistics and ran substantially faster. Across five random seeds, the mean OOF AUROC was 0.7696 for sum features and 0.7648 for the full nine-channel features; the former required roughly one-tenth of the mean runtime.

Feature-importance analysis shows that LightGBM uses about 1,468 features on average across the five folds. Overall importance rankings are highly correlated, but the overlap among the top 30 features is only moderate. Because the data do not provide clinical variable names, this project cannot offer medical or causal interpretations of those features.

Threshold analysis showed that the default threshold of 0.5 identified only about 21.5% of positive samples. Lowering the threshold can increase sensitivity but also produces more false positives, so any deployment would need to select a decision threshold based on the costs of missed diagnoses and false positives in the relevant medical setting.

The main contribution of this project is not to establish that the model is ready for clinical use, but to develop a complete research workflow encompassing automated modeling, human review, fair comparison, robustness assessment, feature interpretation, and error analysis.

Contents

Abstract 2
1. Research Background 5
2. Data and Prediction Task 5
2.1 Data Sources 5
2.2 Data Size 5
2.3 Prediction Objective 6
2.4 Key Evaluation Indicators 6
3. Experimental Environment 6
3.1 Hardware and Systems 6
3.2 Software and Model Services 7
3.3 ARM64-Compatible Processing 7
4. Roles of Automated Tools and Manual Research 7
5. Experiment 001: Verification of the TimesNet Paper Implementation 8
5.1 Study Purpose 8
5.2 Results 8
5.3 Conclusion 9
6. Experiment 002: Manually Developed Logistic-Regression Baseline 9
6.1 Feature Processing 9
6.2 Model and Validation Method 9
6.3 Results 9
6.4 Explanation 10
7. Experiment 003: RD-Agent-Generated ARF Prediction Pipeline 10
7.1 Automatically Generated Pipeline 10
7.2 Final Results After Resuming the Run 10
7.3 RAG Issue and Recovery 11
7.4 Limitations of the Submitted File 11
8. Experiment 004: Fair Model Comparison 12
8.1 Study Purpose 12
8.2 Results 12
8.3 Conclusion 12
9. Experiment 005: Feature-Ablation Experiment 13
9.1 Study Purpose 13
9.2 Results 13
9.3 Conclusion 13
10. Experiment 006: Multi-Seed Robustness Experiment 13
10.1 Study Purpose 13
10.2 Results 14
10.3 Interpretation 14
11. Experiment 007: Feature-Importance and Stability Analysis 14
11.1 Experimental Setup 14

11.2 Model Results 15
11.3 Interpretation 15
11.4 Interpretation Limitations 15
12. Experiment 008: Thresholds, Calibration, and Error Analysis 16
12.1 Basic Results 16
12.2 Threshold Results 17
12.3 Interpretation 17
12.4 Method Limitations 17
13. Summary of Consolidated Results 18
14. Data-Leakage and Performance Audit 18
14.1 ICU-Stay-Level Duplicate Check 18
14.2 ARF_ONSET_HOUR 19
14.3 Test-Label Leakage Check 19
14.4 Limitations of Independent Test Performance 19
15. Key Findings 20
15.1 Automated modeling can produce effective starting points 20
15.2 LightGBM Achieves Better Internal Validation Performance in a Fair Comparison 20
15.3 More features are not necessarily better 20
15.4 The results are somewhat robust to random partitioning 20
15.5 Feature Importance Is Broadly Stable, but Top Rankings Still Vary 21
15.6 Default thresholds are not appropriate for high-risk screening 21
16. Study Limitations 21
17. Ethical and Clinical Instructions for Use 22
18. Reproducibility and Preservation of Results 22
18.1 Public Release Boundary 22
18.2 Archived Environment Record 22
18.3 Aggregate Results 23
18.4 Figures 23
18.5 Excluded Materials 23
18.6 Using the Archive 23
18.7 Integrity Check 23
19. Final Conclusions 23
20. References 25

1. Research Background

Acute respiratory failure is a serious clinical condition that can worsen rapidly. Predicting the risk of ARF occurring in the next 12 hours from the preceding 12 hours of electronic-health-record data may support early monitoring and resource allocation.

This study is structured around the following questions:

- Does RD-Agent work in ARM64 Ubuntu VMs?
- Can RD-Agent automatically generate a complete clinical time-series prediction pipeline?
- Do automatically generated models outperform simple human-developed baselines?
- Are the performance differences coming from the model itself, rather than an unfair partitioning of the data?
- Do more temporal summary statistics necessarily improve performance?
- Are the experimental results stable across different random data splits?
- What are the main features that LightGBM relies on?
- How do different classification thresholds affect missed diagnoses and false positives?
- Does the modeling process leak test-set labels into training?
- Can current results support real-world clinical applications?

2. Data and Prediction Task

2.1 Data Sources

This project uses the publicly released RD-Agent Data Science sample resource:

`arf-12-hours-prediction-task`

The task resource and its applicable access terms are documented in the official RD-Agent repository, in `docs/scens/data_science.rst`.

RD-Agent Project Source:

`https://github.com/microsoft/RD-Agent`

The supplied task resource contains preprocessed, anonymized ICU-stay-level clinical features. This report does not independently document the upstream clinical database or release version, cohort-selection rules, or the operational process used to construct the ARF label. It therefore evaluates methodological performance on the supplied task resource and does not establish clinical validity for a separately traceable patient cohort.

2.2 Data Size

Training set

- Number of samples: 9,921
- Observation-window length: 12 hours
- Number of features per hour: 4,816
- Original tensor shape: `(9921, 12, 4816)`
- Negative samples: 8,961

- Positive samples: 960
- Positive rate: 9.68%

Test set

- Number of samples: 4,253
- Original tensor shape: `(4253, 12, 4816)`

2.3 Prediction Objective

The model takes multivariate clinical time series from the preceding 12 hours as input and outputs the predicted risk of ARF in the following 12 hours.

Label Definition:

- `0`: No ARF in the next 12 hours
- `1`: ARF in the next 12 hours

This is a class-imbalanced binary-classification task.

2.4 Key Evaluation Indicators

The study uses the following primary evaluation metrics:

- AUROC
- Average Precision (AP)
- Brier score
- Sensitivity / Recall
- Specificity
- Precision
- F1
- Balanced Accuracy

Since the positive rate is only 9.68%, Accuracy alone can be misleading. For example, a model that consistently predicts negative results may also achieve an accuracy of about 90.3%, so AUROC and AP are more suitable for evaluating the sequencing and positive recognition capabilities of the model.

The uninformative reference level of the AP is roughly equal to the positive ratio, that is, about 0.0968.

3. Experimental Environment

3.1 Hardware and Systems

- Host: Apple Silicon M1 Mac
- Virtualization tool: UTM
- VM System: Ubuntu 24.04 LTS
- CPU architecture: ARM64/aarch64
- VM Memory: 6 GB
- Virtual Disk: Approximately 100 GB

- Python: 3.10.20

3.2 Software and Model Services

- RD-Agent: Source Code Installation
- Chat model: MiniMax-M3
- NumPy:2.2.6
- pandas:2.3.3
- scikit-learn:1.7.2
- SciPy:1.14.1
- LightGBM:4.7.0
- matplotlib:3.10.9

The exact Git commit of RD-Agent is retained in the public archive at:

`archive/arf_v1/environment.txt`

The authoritative commit hash saved there is `4f9ecb005881cddc08df0124a2e894c018007679`.

3.3 ARM64-Compatible Processing

Part of RD-Agent's data science Docker environment relies on x86_64 Kaggle images and cannot be used directly in the current ARM64 virtual machine.

To avoid this architectural limitation, this project switches the Data Science Coding and Runner environments to Conda:

```
DS_Coder_CoSTEER_env_type=conda
DS_Runner_CoSTEER_env_type=conda
```

A separate `kaggle` Conda environment is then established and verified:

- Python executable available
- `coverage` available
- `strace` available
- Data directory is accessible
- RD-Agent able to execute generated code in the local Conda environment

This enables the experiment to continue without relying on x86_64 Docker images.

4. Roles of Automated Tools and Manual Research

This project does not treat RD-Agent output as a final research conclusion.

RD-Agent is primarily responsible for:

- Read the task description
- Generate data-loading code
- Generate feature-engineering workflows
- Train LightGBM
- Perform five-fold cross-validation

- Generate OOF predictions
- Generate test-set predictions and submission files
- Use execution feedback to repair code

Manual research work mainly includes:

- Check data structure and label distribution
- Establish a logistic-regression baseline
- Audit for test-label leakage
- Standardize the model-validation protocol
- Re-run a fair comparison
- Design feature-ablation experiments
- Design multi-seed robustness experiments
- Analyze feature-importance stability
- Analyze classification thresholds, calibration, and error cases
- Save code, logs, software versions, and checksums
- Distinguish internal OOF results from unscored test-set predictions and any future independent test-set evaluation

This is therefore a study of automated modeling combined with human methodological review, rather than a one-time execution of automated commands.

5. Experiment 001: Verification of the TimesNet Paper Implementation

5.1 Study Purpose

This experiment verifies that RD-Agent can:

- Read the TimesNet paper
- Interpret the model architecture
- Automatically generate PyTorch code
- Execute generated code
- Revise code in response to execution feedback

Paper:

TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis

Paper URL:

`https://arxiv.org/abs/2210.02186`

5.2 Results

RD-Agent successfully generated and executed TimesNet model code, and the output tensor shape is:

```
(8, 1)
```

The final evaluation is:

```
SUCCESS
```

The experiment was completed in the ARM64 Ubuntu virtual machine, and there was no incompatibility with the arm image.

On the first run, the execution environment did not resolve Python correctly. After re-entering the correct Conda environment, the code executed successfully.

5.3 Conclusion

This experiment verifies an end-to-end workflow of paper parsing, code generation, and execution by RD-Agent.

However, this result only indicates that the code meets the current interface and execution evaluation requirements, and does not represent:

- Full reproduction of the TimesNet paper
- Complete training on real data
- Predictive performance reported in the paper
- Rigorous external validation

6. Experiment 002: Manually Developed Logistic-Regression Baseline

6.1 Feature Processing

For each sample, the same features are summed over 12 hours, converting the original tensor:

```
(9921, 12, 4816)
```

to:

```
(9921, 4816)
```

`ARF_ONSET_HOUR` was not used as a feature to avoid direct label leakage.

6.2 Model and Validation Method

Model:

- Logistic regression with class-balanced weights

Data partitioning:

- 80% training
- 20% validation
- Stratified random split
- `random_state=42`

6.3 Results

- AUROC: 0.7138
- AP: 0.2487
- Accuracy: 0.8196
- Confusion matrix: `[[1551, 242], [116, 76]]`
- Positive recall: approximately 39.6%

- Precision (positive predictive value): about 23.9%

6.4 Explanation

This model provides a simple manually developed baseline for subsequent analyses.

Because this experiment uses only one training/validation split whereas RD-Agent uses five-fold cross-validation, its AUROC of 0.7138 is not strictly comparable with subsequent RD-Agent results.

A truly fair comparison is done in Experiment 004.

7. Experiment 003: RD-Agent-Generated ARF Prediction Pipeline

7.1 Automatically Generated Pipeline

The pipeline generated by RD-Agent includes:

- FIDDLE sparse-tensor loading
- Block-wise feature processing
- Nine types of time statistics
- Five-fold stratified cross-validation
- LightGBM binary classification
- Class-imbalance weights
- Early stopping
- OOF prediction
- Test-set prediction
- Submission-file generation

Nine types of features include:

- sum
- max
- mean
- std
- min
- first
- last
- slope
- nonzero

The complete feature dimensions are:

```
4816 × 9 = 43344
```

7.2 Final Results After Resuming the Run

Fold-level AUROC:

- Fold 0: 0.7537
- Fold 1: 0.7558

- Fold 2: 0.7814
- Fold 3: 0.7867
- Fold 4: 0.7931

Pooled AUROC across all OOF predictions:

```
0.7615
```

Here, 0.7615 is the pooled OOF AUROC, not the arithmetic mean of fold-level AUROCs.

7.3 RAG Issue and Recovery

The model code, cross-validation, and submission files have all been successfully generated.

The first run encountered an embedding-service error during the automatic RAG experience-writing phase. This error occurred in the optional knowledge-management phase, not in model training or prediction.

Automatic knowledge generation was then disabled for Data Science Coding and Runner, and the run resumed from the Coding checkpoint. RD-Agent then completed:

- Execution
- Feedback
- Record
- Submission-format checking

The program exit code is:

```
0
```

The problem did not affect this round of model training, cross-validation, or prediction, but its experience record was not fully written to the RAG knowledge base.

7.4 Limitations of the Submitted File

The RD-Agent source code first calculates continuous predicted probabilities for the test set:

```
test_probs = test_preds_sum / N_FOLDS
```

The probability is then converted to a binary label using the threshold:

```
test_labels = (test_probs >= optimal_threshold).astype(np.int32)
```

The final submission is a binary label.

Because the scorer uses AUROC, continuous probabilities generally retain more ranking information than binary labels. Subsequent formal submissions should instead use:

```
submission_df = pd.DataFrame({
    "ID": ids_test,
    "ARF_LABEL": test_probs,
})
```

To preserve the traceability of the original experiment, this project did not overwrite RD-Agent's archived results.

8. Experiment 004: Fair Model Comparison

8.1 Study Purpose

This experiment resolves the mismatch between the validation protocols used in Experiments 002 and 003.

It evaluates both logistic regression and LightGBM using:

- Same 12-hour summary features
- Same five-fold stratified cross-validation
- Same random seed
- Same training sample
- Identical validation samples
- Same evaluation metrics

8.2 Results

Model	Feature set	Mean fold-level AUROC	Fold-level AUROC SD	Pooled OOF AUROC	Mean fold-level AP	Pooled OOF AP
Logistic Regression	12h sum	0.6981	0.0222	0.6978	0.2388	0.2332
LightGBM	12h sum	0.7805	0.0242	0.7770	0.3269	0.3173

Difference in pooled OOF AUROC:

```
0.7770 - 0.6978 = 0.0792
```

8.3 Conclusion

With the same data partition, summary features, and evaluation protocol, LightGBM achieved higher AUROC and AP than logistic regression.

Because the data partition and input features were controlled, the result suggests that model choice and nonlinear representation may contribute to the performance difference. However, hyperparameters, regularization, and optimization may also matter, so the improvement cannot be attributed solely to model expressiveness.

Possible explanations include:

- LightGBM is able to learn nonlinear relationships;
- LightGBM can automatically model the interaction between features;
- LightGBM may be more appropriate for the high-dimensional clinical features in this study;
- Logistic regression mainly learns linear decision boundaries, and its expression ability is relatively limited.

By standardizing the data partition, input features, and evaluation metrics, this experiment provides the project's principal fair model comparison.

9. Experiment 005: Feature-Ablation Experiment

9.1 Study Purpose

Compare simple and complex features:

- `sum`: 12-hour sum of 4,816 features
- `all_9_channels`: nine statistical features with a total of 43,344 features

Both scenarios use the same five-fold split and the same LightGBM settings.

9.2 Results

Feature scheme	Pooled OOF AUROC	Pooled OOF AP	Number of features	Relative runtime
sum	0.7775	0.3155	4,816	1.0×
all_9_channels	0.7698	0.3090	43,344	Approx. 8.4 ×

9.3 Conclusion

The full nine-channel feature representation has about nine times as many features as the summed representation, but did not yield higher pooled OOF AUROC or AP.

Possible causes include:

- Many statistical features are redundant
- The sample size is insufficient to support 43,344-dimensional features
- Some statistics introduce noise
- The sum feature already contains the main predictive signal
- LightGBM's feature filtering does not fully eliminate high-dimensional noise

Therefore, on the current data, simple sum features offer a better performance-efficiency trade-off.

10. Experiment 006: Multi-Seed Robustness Experiment

10.1 Study Purpose

This experiment tests whether the conclusion of Experiment 005 depends on a particular random split.

Random Seeds

- 7
- 21
- 42
- 77
- 2026

Each seed undergoes 5-fold stratified cross-validation.

Both feature representations use exactly the same data partition for each random seed.

10.2 Results

Feature scheme	Mean OOF AUROC	Standard deviation	Minimum	Maximum	Mean OOF AP	Mean runtime
sum	0.7696	0.0070	0.7626	0.7775	0.3110	23.1 seconds
all_9_channels	0.7648	0.0062	0.7541	0.7698	0.3108	233.0 seconds

Mean Paired AUROC Difference:

```
sum - all_9_channels = +0.0048
```

The summary-feature scheme achieved a higher OOF AUROC in four of the five random seeds.

10.3 Interpretation

The mean AUROC advantage of the sum feature is small and should not be interpreted as evidence of a practically meaningful performance difference.

However, the average running time of the full nine-channel feature is about ten times that of the sum feature, while the average AP is almost the same.

A more robust conclusion is therefore that:

> Under the current data, models, and random seeds, simple sum features achieve predictive performance no worse than full nine-channel features while substantially reducing computational cost.

This experiment used only five random seeds and did not perform a formal significance test, so it provides limited repeated validation.

11. Experiment 007: Feature-Importance and Stability Analysis

11.1 Experimental Setup

The experimental setup used:

- 12-hour summary features
- LightGBM
- Five-fold stratified cross-validation
- Random seed 42
- Gain-based importance

LightGBM parameters include:

```
objective = binary
metric = auc
learning_rate = 0.01
num_leaves = 63
max_depth = -1
min_data_in_leaf = 50
feature_fraction = 0.8
bagging_fraction = 0.8
bagging_freq = 5
lambda_l1 = 0.1
lambda_l2 = 0.1
num_boost_round = 8000
early_stopping_rounds = 100
```

Each fold calculates `scale_pos_weight` based on the training data.

11.2 Model Results

- Pooled OOF AUROC: 0.7775
- Pooled OOF AP: 0.3155
- Mean fold-level AUROC: 0.7835 ± 0.0220
- Mean fold-level AP: 0.3265 ± 0.0458
- Average number of features used per fold: approximately 1,468
- Number of features used in all five folds: 1,093
- Spearman correlation of all features outside the top 30: approximately 0.8827
- Top 30 average Jaccard: approximately 0.4479

11.3 Interpretation

The importance rankings of all features have a high correlation across folds, indicating that the model's overall signal is relatively stable.

However, the Jaccard of the Top 30 features is only about 0.448, indicating that the most important features will still change with the sample division.

Therefore, a top-ranked feature from a single fold cannot be directly interpreted as a stable clinical determinant.

11.4 Interpretation Limitations

The current data provide only anonymized feature indices:

```
feature_0
feature_1
...
feature_4815
```

True clinical variable names corresponding to these characteristics were not provided.

Therefore, this project can only analyze:

- Feature Index
- Importance Size
- Cross-fold stability
- Statistical differences between positive and negative samples
- Non-zero incidence

The project therefore cannot provide:

- Clinical-variable interpretation
- Physiological-mechanism explanations
- Medical causal inference
- Clinical decision-making recommendations

12. Experiment 008: Thresholds, Calibration, and Error Analysis

12.1 Basic Results

Data:

- Number of samples: 9,921
- Number of positive cases: 960
- Positive rate: 0.0968

Model:

- OOF AUROC:0.7775
- OOF AP:0.3155
- Brier score:0.0823
- Average prediction probability: 0.1458

The model's mean predicted probability exceeds the observed positive-class prevalence, suggesting overall overprediction. However, without an independent calibration set, this difference alone cannot establish calibration quality.

12.2 Threshold Results

Threshold scheme	Threshold value	Sensitivity	Specificity	Precision	False Positive	False Negative
Fixed Threshold	0.5000	0.2146	0.9711	0.4430	259	754
Youden J	0.1738	0.6677	0.7573	0.2276	2,175	319
Max F1	0.2443	0.5271	0.8478	0.2706	1,364	454
Target sensitivity 80%	0.1061	0.8000	0.5990	0.1761	3,593	192
Target sensitivity 90%	0.0626	0.9000	0.4204	0.1426	5,194	96

12.3 Interpretation

A threshold is a dividing line that converts a continuous risk probability into a 0/1 classification result.

For example:

```
Predicted probability >= threshold -> predict ARF
Predicted probability < threshold -> predict no ARF
```

The default threshold of 0.5 has a high specificity, but only about 21.5% of positive samples will be found, and 754 positive cases will be missed.

Lowering the threshold reduces missed diagnoses:

- Increased sensitivity
- False Negative Decrease
- Increased false positives
- Precision Reduction

Medical screening is often more focused on reducing missed diagnoses, so a lower threshold may be chosen. But lower thresholds lead to more false positives, additional examinations, and clinical workload.

12.4 Method Limitations

All thresholds were selected and evaluated using the same set of OOF predictions, so these results are exploratory internal analyses.

Stricter deployment studies should:

- Select thresholds using training or development data;
- Evaluate fixed thresholds on an independent validation set or external data;
- Determine thresholds using real clinical costs;
- Calibrate predicted probabilities; and
- Analyze sensitivity and specificity across patient subgroups.

13. Summary of Consolidated Results

Experiment ID	Model or task	Validation design	AUROC	AP	Notes
001	TimesNet code implementation	Interface and execution validation	N/A	N/A	Established that the code pipeline operated end to end
002	Logistic regression	Single 80/20 split	0.7138	0.2487	Preliminary human-developed baseline
003	RD-Agent LightGBM	Five-fold OOF	0.7615	Not reported	Automated-modeling result; AP was not uniformly documented
004	Logistic regression	Fair five-fold OOF	0.6978	0.2332	Fair baseline
004	LightGBM	Fair five-fold OOF	0.7770	0.3173	Same summary features
005	LightGBM sum	Five-fold OOF	0.7775	0.3155	4,816 dimensions
005	LightGBM all 9	Five-fold OOF	0.7698	0.3090	43,344 dimensions
006	LightGBM sum	Five-seed average	0.7696	0.3110	Robustness experiment
006	LightGBM all 9	Five-seed average	0.7648	0.3108	Approximately ten times the running time
007	LightGBM sum	Five-fold OOF	0.7775	0.3155	Feature-importance analysis
008	LightGBM sum	Same OOF set	0.7775	0.3155	Threshold and error analysis

Requires attention

- Experiment 002's value of 0.7138 comes from a single split and cannot be directly compared with five-fold OOF results.
- Experiment 004 is the fairest model comparison of logistic regression with LightGBM.
- Experiments 005, 007, and 008 use the same or corresponding five-fold OOF predictions and should not be considered three independent performance replications.
- Experiment 006 is the repeated robustness verification under different random divisions.
- There is currently no independent test set AUROC, so all reported performance is the result of internal cross-validation of the training data.

14. Data-Leakage and Performance Audit

14.1 ICU-Stay-Level Duplicate Check

Training Set

- Total samples: 9,921
- Number of unique IDs: 9,921
- Duplicate IDs: 0

Accordingly, no available ICU-stay ID appeared in multiple validation folds.

However, the data do not provide a patient-level SUBJECT_ID, so it is not possible to rule out multiple hospitalizations for the same patient appearing across folds.

14.2 ARF_ONSET_HOUR

ARF_ONSET_HOUR is directly associated with the label.

The human-developed baseline and subsequent fair-comparison experiments did not use this field as a predictive feature, thereby avoiding direct target leakage.

14.3 Test-Label Leakage Check

The program reads the test set metadata file, but only uses the test sample ID in the modeling phase:

```
ids_test = test_meta["ID"].values
```

The true test ARF_LABEL was not read or used in:

- Model training
- Five-fold cross-validation
- Threshold selection
- Feature selection
- Hyperparameter tuning

In the code:

```
test_labels = (test_probs >= optimal_threshold).astype(np.int32)
```

Here, `test_labels` are the binary predicted labels generated by the model based on the predicted probabilities, not the actual test labels. The variable name is potentially misleading, but it does not affect the program logic.

The true test labels are located in the independent scoring directory and are used by `grade.py` after prediction is complete.

Therefore, this audit did not find any cases of direct training or parameter tuning using the real label of the test set.

14.4 Limitations of Independent Test Performance

The test set in the public modeling catalog does not contain real labels that can be evaluated directly by the investigator.

Therefore, no independent test set AUROC, AP or sensitivity was reported for this project.

All current key performance conclusions are derived from:

- Five-fold cross-validation
- OOF prediction
- Internal replication across multiple random seeds

These results can compare models and feature schemes, but cannot substitute for validation at an independent hospital, in an independent time period, or in an external cohort.

15. Key Findings

15.1 Automated modeling can produce effective starting points

RD-Agent completed successfully:

- Data loading
- Feature engineering
- Model training
- Five-fold cross-validation
- OOF prediction
- Test-set prediction
- Submission-file generation
- Feedback and remediation

This shows that automated R&D agents can generate usable data-science starting points relatively quickly.

However, the output still needs to be manually checked to verify the design, data leakage, evaluation indicators, and submission format.

15.2 LightGBM Achieves Better Internal Validation Performance in a Fair Comparison

Under the same data split, sum features, and evaluation metrics:

- Logistic-regression OOF AUROC: 0.6978
- LightGBM OOF AUROC:0.7770

In this experiment, LightGBM achieved better internal validation performance on the current task.

15.3 More features are not necessarily better

The full nine-channel feature representation has roughly nine times as many dimensions as the sum-feature representation, yet it does not improve mean AUROC or AP across the five random seeds and increases runtime by roughly a factor of ten.

This indicates that:

- High-dimensional features may be redundant;
- Simple features may already contain the main signal;
- Feature complexity should be tested through ablation; and
- Model complexity and computational cost do not automatically imply better performance.

15.4 The results are somewhat robust to random partitioning

Across five random seeds, sum features achieved a higher OOF AUROC in four.

Although the mean difference is only 0.0048, the sum-feature scheme has substantial computational-efficiency advantages.

Therefore, the sum feature is the currently more plausible default scheme.

15.5 Feature Importance Is Broadly Stable, but Top Rankings Still Vary

Feature-importance rankings have high overall Spearman correlation across five folds, but the Top-30 Jaccard overlap is only moderate.

This indicates that:

- The model's overall signal is relatively stable;
- The ordering of individual top features remains sensitive to the data split; and
- A single training run should not support definitive medical interpretation.

15.6 Default thresholds are not appropriate for high-risk screening

The sensitivity of the threshold of 0.5 is only about 21.5%, and most positive cases will be missed.

If the goal is to screen high-risk patients early, a lower threshold needs to be considered, but more false positives must be accepted.

Therefore, model performance and decision thresholds are two different issues:

- AUROC and AP assess discriminative ranking performance.
- For a given set of predictions, decision thresholds determine the trade-off between false positives and false negatives.

16. Study Limitations

- The data do not provide true clinical feature names, so features cannot be interpreted as medical variables.
- The data do not provide patient-level `subject_ID`, and the possibility of multiple hospitalizations for the same patient across folds cannot be completely ruled out.
- There are currently no independent test sets or external hospital validation results.
- All key performances are internal cross-validation results.
- The data consist of structured features processed by FIDDLE and do not constitute a complete original electronic medical record.
- Multi-random seed experiments used only five seeds and no formal statistical significance test was performed.
- Experiments 007 and 008 used the same underlying OOF predictions rather than independently replicated experiments.
- Thresholds were selected and evaluated on the same set of OOF predictions, so some optimism bias may be present.
- Probabilistic calibration model comparisons, such as Platt scaling or isotonic regression, were not performed.
- Equity was not analyzed for patient subgroups such as age, gender, ethnicity, disease type, etc.
- No out-of-time or cross-institutional validation was performed.
- No real-world clinical utility, workload, and cost analyses were performed.
- RD-Agent's RAG Knowledge Write feature was disabled because of long-text embedding restrictions.

- ARM64 environments use Conda instead of the default Docker, reducing isolation and the ability to fully reproduce the environment.
- Model results cannot be used directly for medical diagnosis or clinical decision making.

17. Ethical and Clinical Instructions for Use

This project is for data science research and educational demonstrations only.

Model prediction results cannot be used for:

- Actual medical diagnosis
- Patient treatment decisions
- Automatic allocation of ICU resources
- Replacement of physician judgment
- Unvalidated clinical deployment

Before any real clinical use, the model would require:

- Independent external validation;
- Cross-hospital validation;
- Out-of-time validation;
- Subgroup equity assessment;
- Probability calibration;
- Clinical-utility analysis;
- Human-computer collaborative design with clinician involvement;
- Privacy and data-security review; and
- Regulatory and ethical approval.

18. Reproducibility and Preservation of Results

18.1 Public Release Boundary

The public repository retains selected code, aggregate tables, figures, the environment record, and checksum records needed to inspect this study. It does not include the original per-experiment `learning_records`, raw clinical data, intermediate feature caches, individual-level predictions, complete execution logs, or generated submission files.

18.2 Archived Environment Record

`archive/arf_v1/environment.txt`

18.3 Aggregate Results

`archive/arf_v1/results/fair_comparison_summary.csv`

`archive/arf_v1/results/feature_ablation_summary.csv`

`archive/arf_v1/results/multiseed_summary.csv`

`archive/arf_v1/results/feature_importance_top30.csv`

`archive/arf_v1/results/threshold_analysis_summary.csv`

`archive/arf_v1/results/threshold_metrics.csv`

18.4 Figures

`archive/arf_v1/figures/feature_importance_top30.png`

`archive/arf_v1/figures/threshold_analysis.png`

18.5 Excluded Materials

The excluded materials contain source-data derivatives, individual-level outputs, or internal execution records. Their exclusion preserves the public data boundary described in the repository README.

18.6 Using the Archive

The archive supports inspection of the experimental methodology and reported aggregate findings. The archived scripts are not a standalone runtime because they depend on the excluded inputs and intermediate records.

18.7 Integrity Check

The public release records SHA-256 hashes for published reports and selected aggregate artifacts. Run `python scripts/verify_public_release.py` from the repository root to verify the released materials.

19. Final Conclusions

The project completed an end-to-end research process, from environment setup and automated modeling to human-developed baselines, methodological auditing, and robustness analysis.

RD-Agent successfully generated and ran the 12-hour ARF prediction pipeline in an ARM64 Ubuntu Conda environment. Subsequent human review found that:

- Under the same five-fold partition and summary features, LightGBM achieved a pooled OOF AUROC of 0.7770, higher than logistic regression's 0.6978.
- LightGBM's pooled OOF AP was 0.3173, higher than logistic regression's 0.2332.
- The full nine-channel feature representation did not consistently outperform the simpler summary-feature representation.
- Summary features achieved similar or better performance in multi-seed experiments while requiring roughly one-tenth of the runtime.
- Overall feature-importance rankings were highly stable, although the highest-ranked features remained sensitive to data partitioning.
- The default threshold of 0.5 missed most positive samples; threshold selection must account for the costs of missed diagnoses and false positives.
- No direct training or hyperparameter tuning using true test-set labels was identified.
- The current results are internal cross-validation results and cannot be interpreted as independent test performance or clinical effectiveness.
- Automated agents can improve modeling efficiency, but do not replace human study design, leakage audits, metric interpretation, or clinical risk judgment.

Therefore, the most important conclusion of this project is not that "RD-Agent automatically generates a high-scoring model", but that:

> Automated R&D agents can generate actionable data science solutions, but reliable research results still rely on establishing fair baselines, checking data leaks, conducting ablation and

robustness experiments, analyzing model errors, and honestly explaining the limitations of external verification and clinical interpretation.

20. References

1. Microsoft. RD-Agent [Computer software]. GitHub. https://github.com/microsoft/RD-Agent

2. Wu, H., Hu, T., Liu, Y., Zhou, H., Wang, J., & Long, M. (2023). TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis. International Conference on Learning Representations. https://openreview.net/pdf?id=ju_Uqw384Oq

3. Ke, G., Meng, Q., Finley, T., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. Advances in Neural Information Processing Systems, 30. https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html

4. Microsoft RD-Agent project. Data Science documentation [Documentation]. https://github.com/microsoft/RD-Agent/blob/main/docs/scens/data_science.rst

5. scikit-learn developers. Metrics and scoring: Quantifying the quality of predictions [Documentation]. https://scikit-learn.org/stable/modules/model_evaluation.html

6. Tang, S., Davarmanesh, P., Song, Y., et al. (2020). Democratizing EHR Analyses with FIDDLE: A Flexible Data-Driven Preprocessing Pipeline for Structured Clinical Data. Journal of the American Medical Informatics Association, 27(12), 1921-1934. https://doi.org/10.1093/jamia/ocaa139